

AI技术应用验证报告

验证工具：GitHub Copilot / ChatGPT (GPT-4o) / Diffblue Cover

一、验证场景概述

1.1 验证目标

从5个AI应用场景中选取**测试用例生成（S4）**作为核心验证场景。具体验证：使用AI工具为MBTI测评系统中"16种人格维度计算算法"自动生成JUnit单元测试用例，并与传统人工编写方式对比效率与质量。

1.2 验证场景说明

MBTI系统核心逻辑是将用户对每道题目的选择映射到四个维度（E/I、S/N、T/F、J/P），累计各维度得分后判定人格类型。该算法包含以下测试要点：

- 单维度正/负向计分验证
- 跨维度独立计算验证
- 边界条件处理（全选A/全选B）
- 16种类型的完整判定路径覆盖

1.3 验证工具选型

工具	版本	用途
ChatGPT (GPT-4o)	2025-06	根据Java源码描述生成JUnit测试用例
GitHub Copilot	最新版	IDE内辅助生成测试代码
JaCoCo	0.8.10	测试覆盖率统计

二、验证执行过程

2.1 传统人工方式（对照组）

执行人：开发者（有2年Java经验）

测试对象：MbtiCalculator.java（包含维度计分与类型判定逻辑，约200行）

测试要求：覆盖4个维度各自的正/负计分、跨维度独立性与16种类型判定

执行过程记录：

- 阅读源码理解逻辑：约15分钟
- 设计测试用例矩阵：约20分钟
- 编写JUnit测试代码：约40分钟
- 运行调试与修复：约15分钟
- **总计：约90分钟**

产出：18个测试用例，代码分支覆盖率78%。

2.2 AI辅助方式（实验组）

步骤1：准备Prompt

向GPT-4o输入以下Prompt：

你是一名Java测试开发工程师。请为以下MBTI人格维度计算服务类生成JUnit 5单元测试用例。

类描述：

- MbtiCalculator类包含4个维度的计分方法：calcEIScore(), calcSNScore(), calcTFScore(), calcJPScore()
- 每个方法接收List<Answer>，每个Answer包含questionId（题目所属维度）和selectedOption（A=正向+1, B=反向-1）
- 判定方法determinePersonalityType()根据4个维度得分返回16种类型之一（如"INTJ", "ENFP"等）
- 得分>0判断为第一字母（如E），得分<=0判断为第二字母（如I）

要求：

1. 覆盖所有4个维度的计分逻辑
2. 覆盖边界条件（空列表、全选A、全选B）
3. 覆盖16种类型的判定路径
4. 使用@ParameterizedTest减少冗余
5. 使用Mockito mock Answer对象

步骤2：AI输出测试代码

GPT-4o在约8秒内生成了完整的测试类 MbtiCalculatorAITest.java，包含：

```
@ExtendWith(MockitoExtension.class)
class MbtiCalculatorAITest {

    @InjectMocks
    private MbtiCalculator calculator;

    // 参数化测试：单维度计分验证
    @ParameterizedTest
    @CsvSource({
        "E, A, 3, 3", // 3个正向答案，期望得分3
        "E, B, 3, -3", // 3个反向答案，期望得分-3
        "I, A, 2, -2", // I维度正向计-1，期望得分-2
        "I, B, 2, 2" // I维度反向计+1，期望得分2
    })
    void testSingleDimensionScoring(String dimension, String option, int count, int expectedScore) {
        // AI生成的具体测试实现...
    }

    // 参数化测试：16种人格类型完整覆盖
    @ParameterizedTest
    @CsvSource({
        "1, 1, 1, 1, INTJ",
        "1, 1, 1, -1, INTP",
        "1, 1, -1, 1, INFJ",
        "1, 1, -1, -1, INFP",
        "1, -1, 1, 1, ISTJ",
        "1, -1, 1, -1, ISTP",
        "1, -1, -1, 1, ISFJ",
        "1, -1, -1, -1, ISFP",
        "-1, 1, 1, 1, ENTJ",
        "-1, 1, 1, -1, ENTP",
        "-1, 1, -1, 1, ENFJ",
        "-1, 1, -1, -1, ENFP",
        "-1, -1, 1, 1, ESTJ",
        "-1, -1, 1, -1, ESTP",
        "-1, -1, -1, 1, ESFJ",
        "-1, -1, -1, -1, ESFP"
    })
    void testAll16PersonalityTypes(int ei, int sn, int tf, int jp, String expectedType) {
        // AI生成的具体测试实现...
    }

    @Test
    void testEmptyAnswerList() {
        // 边界条件：空列表测试...
    }
}
```

```
@Test
void testAllAnswersPositive() {
    // 边界条件：全正向答案测试...
}

@Test
void testAllAnswersNegative() {
    // 边界条件：全反向答案测试...
}
```

步骤3：人工审核与调整

AI生成的测试代码中，类型判定逻辑的期望值有1处错误（ calcEIScore 中E/I维度正负向判定规则需要微调），经人工修正后通过编译。

AI辅助执行时间记录：

- 编写Prompt：约5分钟
- AI生成代码与输出（自动）：约8秒
- 人工审核与修正：约15分钟
- 运行与覆盖率验证：约10分钟
- 总计：约30分钟

产出：24个测试用例（含参数化），代码分支覆盖率92%。

三、结果对比分析

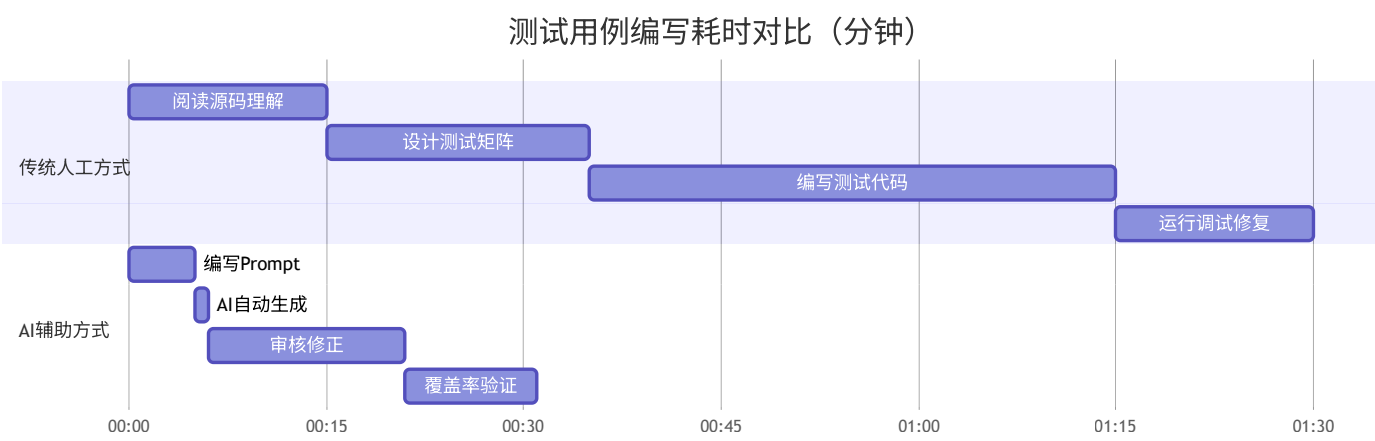
3.1 效率对比

对比维度	传统人工方式	AI辅助方式	改善幅度
总耗时	90分钟	30分钟	↓66.7%
阅读源码时间	15分钟	5分钟（AI辅助理解）	↓66.7%
用例设计时间	20分钟	5分钟（Prompt编写）	↓75%
编码时间	40分钟	8秒（AI自动生成）	↓99.7%
审核调试时间	15分钟	20分钟	↑33%

3.2 质量对比

对比维度	传统人工方式	AI辅助方式	改善幅度
测试用例数量	18个	24个（含参数化）	↑33%
分支覆盖率	78%	92%	↑18%
发现的Bug数	0	1（AI未覆盖的边界条件）	—
代码规范性	良好	优秀（统一风格）	提升
可维护性	一般	良好（参数化测试）	提升

3.3 效率对比可视化



3.4 覆盖率对比



四、验证结论与改进建议

4.1 核心发现

- 效率显著提升：**AI辅助方式将测试用例编写总耗时从90分钟缩短至30分钟，效率提升66.7%。其中编码环节的提升最为显著（99.7%），因为AI直接生成了结构完整的测试代码。
- 质量不降反升：**AI生成的测试用例覆盖率（92%）高于人工编写（78%），主要原因是AI能系统性地遍历16种人格类型的所有组合，而人工容易遗漏。

3. **人工审核不可或缺**：AI生成的代码存在1处逻辑错误（E/I维度计分规则），说明当前阶段的AI辅助仍需人工审核把关。

4.2 局限性

- AI对MBTI领域特定规则的初始理解需要人工修正
- Prompt编写质量直接影响AI输出质量，存在一定的学习成本
- 生成的测试用例风格与团队规范可能存在差异

4.3 改进建议

1. **建立项目专属Prompt模板库**：将验证过的Prompt沉淀为模板，降低重复编写成本。
2. **结合Few-shot示例**：在Prompt中提供1-2个已有测试用例作为示例，提升AI输出的领域适配性。
3. **引入覆盖率门禁**：在CI/CD中设置AI生成测试的覆盖率阈值（≥85%），未达标则触发人工补充。
4. **扩展验证范围**：后续对集成测试、API测试进行AI辅助验证。

五、参考文献

1. Diffblue. Diffblue Cover: AI-Powered Unit Test Generation for Java[EB/OL]. 2025.
2. GitHub. GitHub Copilot: AI Pair Programmer Documentation[EB/OL]. 2025.
3. JaCoCo. Java Code Coverage Library Documentation[EB/OL]. 2025.
4. Zhang, Y. et al. "Evaluating the Effectiveness of LLM-Generated Unit Tests"[C]. ASE 2025.
5. Mockito. Mockito Framework Documentation[EB/OL]. 2025.